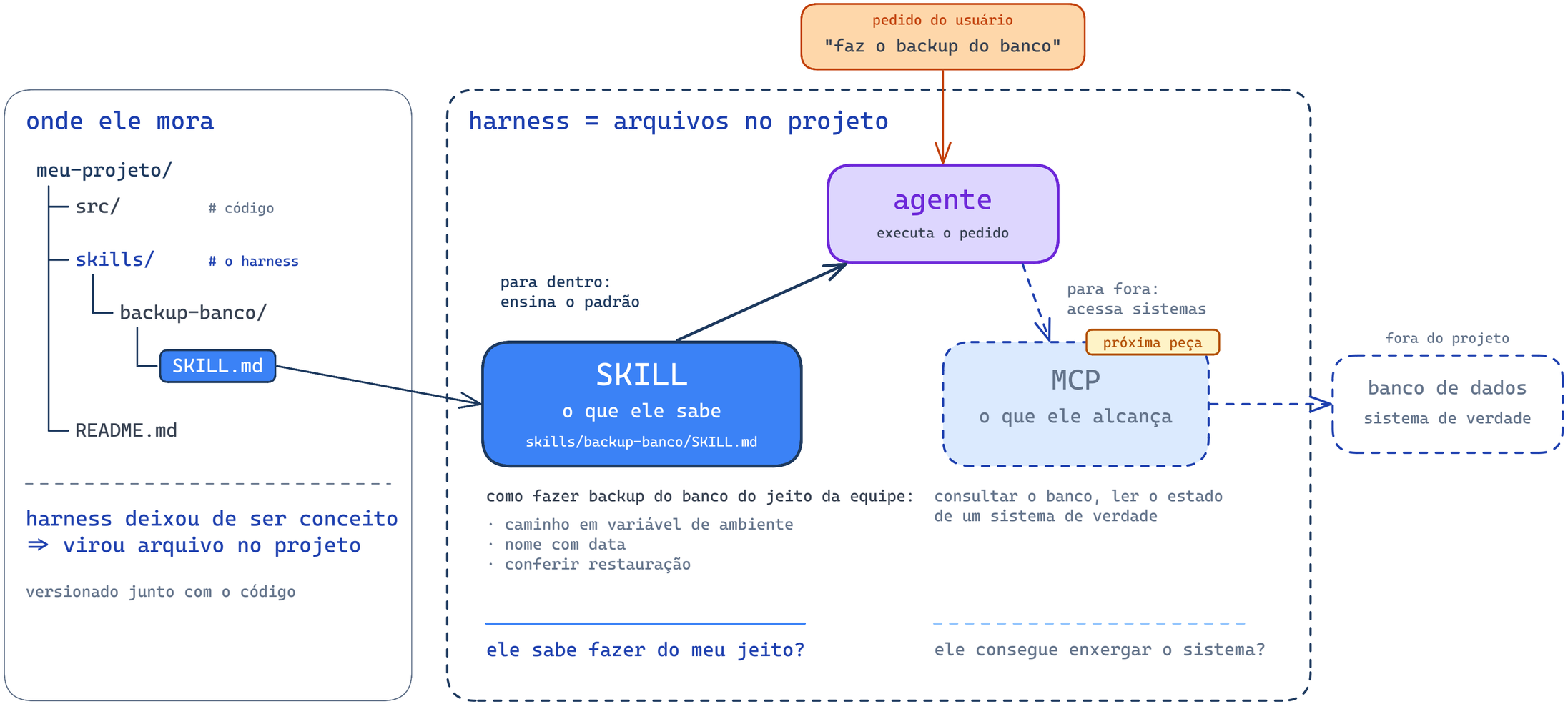
O que o agente sabe e o que ele alcança

skill ⇒ o que ele sabe · MCP ⇒ o que ele alcança



O que é uma skill

Skill: instrução empacotada que ensina o agente a fazer uma tarefa do seu jeito

Pra que serve: baratear a próxima vez

SEM SKILL — explica tudo de novo a cada sessão

- **sessão 1** (explica tudo): Faz o backup do banco. Usa o caminho da variável BACKUP_DIR, coloca a data no nome do arquivo, compacta e confere se restaura.
- **sessão 2** (explica de novo): Faz o backup do banco. Usa o caminho da variável BACKUP_DIR, coloca a data no nome do arquivo, compacta e confere se restaura.
- **sessão 3** (explica de novo): Faz o backup do banco. Usa o caminho da variável BACKUP_DIR, coloca a data no nome do arquivo, compacta e confere se restaura.

3× o mesmo texto, inteiro

custo: a explicação inteira, em toda sessão

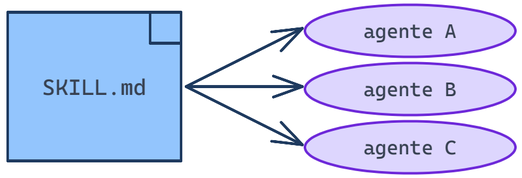
COM SKILL — explica uma vez, depois só pede

- **sessão 1** (explica e empacota): Faz o backup do banco. Usa o caminho da variável BACKUP_DIR, coloca a data no nome do arquivo, compacta e confere se restaura. → skill backup-banco
- **sessão 2** (só pede): Faz o backup do banco. ← a skill aciona e já sabe
- **sessão 3** (só pede): Faz o backup do banco. ← a skill aciona e já sabe

custo: explica uma vez, depois só o pedido

Vantagens

- mesmo resultado entre sessões e entre pessoas da equipe
- padrão cross-agent: o mesmo SKILL.md funciona em agentes diferentes



- só o padrão entra; recurso específico de uma ferramenta fica fora

Quando não usar

"Você precisa de garantia"

⇒ a skill orienta, mas quem executa é o agente, e ele pode pular um passo

"A regra vale para toda tarefa"

⇒ vai no arquivo de instruções do projeto, que o agente lê sempre, não numa skill

As formas de criar uma skill

duas formas, o mesmo SKILL.md

1 À mão

quem escreve: você

você

você escreve cada linha do arquivo

- controle total
- você precisa saber a estrutura do arquivo

---

2 Skill que cria skills

quem escreve: a skill-creator, com você

você

skill-creator

você:
"Quero uma skill para o backup do banco."

skill-creator:
"Quando ela deve entrar?
Que passos ela segue?
Que pedido ela atende?"

⇒ escreve o SKILL.md

- a forma mais usada
- ela conhece a estrutura e pergunta o que falta

SKILL.md

backup-banco/SKILL.md

```
---
name: backup-banco
description: Use ao criar, agendar ou
             restaurar backup do banco
             de dados do projeto.
---

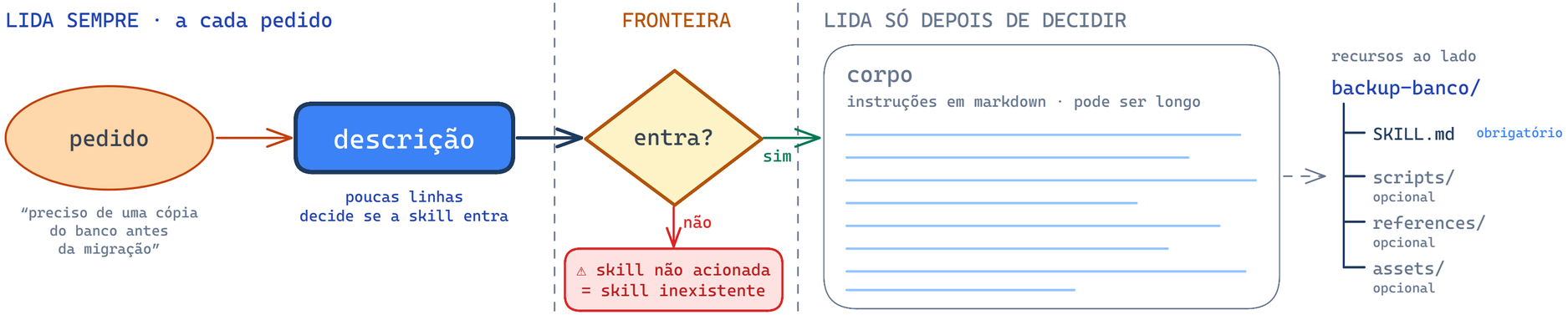
# Backup do banco

1. Leia o caminho de destino da
   variável BACKUP_DIR.
2. Gere o arquivo com a data no nome:
   banco-AAAA-MM-DD.sql.gz
3. Confira que o backup restaura
   antes de dar como pronto.
```

o resultado é o mesmo arquivo

De que uma skill é feita

a descrição roteia · o corpo executa



o SKILL.md por dentro

```
---
name: backup-banco
description: Use ao criar, agendar ou restaurar backup do banco de
  dados do projeto, ou quando o pedido falar em dump, cópia de
  segurança ou recuperar dados.
---

# Backup do banco
1. Leia o caminho de destino da variável BACKUP_DIR.
2. Gere o arquivo com a data no nome: banco-AAAA-MM-DD.sql.gz
3. Compacte e registre o tamanho do arquivo.
4. Confira que o backup restaura antes de dar como pronto.
```

frontmatter: name e description
- lida sempre

corpo em markdown
- lida só depois de decidir

mesmo pedido, duas descrições

"preciso de uma cópia do banco antes da migração"

tímida

```
description:
  Ajuda com backups.
```

✗ não aciona

clara

```
description:
  Use ao criar, agendar ou
  restaurar backup do banco
  ... dump, cópia de
  segurança ...
```

✓ aciona

a descrição é escrita para o momento de acionamento, não para o leitor humano

O critério de empacotamento

o COMO recorrente vira skill · o resto fica no projeto

ENTRADA · itens de uma sessão de trabalho

TESTE DE REAPROVEITAMENTO

DESTINOS

QUE foi feito:
"fiz o backup do banco ontem"

COMO foi feito:
"variável de ambiente, data no nome,
restauração conferida"

⇒ a unidade da skill é o COMO

1. Backup sempre com o caminho na variável BACKUP_DIR e a data no nome
2. Conferir que o backup restaura antes de dar como pronto
3. O banco desta API roda na porta 5433
4. Guardar os backups por 7 dias neste servidor
5. Renomear a tabela pedidos para pedidos_antigos



SKILL

o COMO recorrente

1. caminho em BACKUP_DIR, data no nome
2. conferir que o backup restaura

reaproveitamento em qualquer projeto

FICA NO PROJETO

decisão desta aplicação

3. banco desta API na porta 5433
4. backups por 7 dias neste servidor

comando que se escreve uma vez

5. renomear pedidos para pedidos_antigos

não se repete fora deste projeto

no seu trabalho real

1. liste o que você fez numa sessão
2. separe o QUE do COMO
3. passe cada COMO pelo teste antes de criar a skill

Destilar × declarar

destilar ⇒ o que a IA fez · declarar ⇒ o que deveria ter sido feito

o que volta da sessão



evidência: a sessão do script de backup

```
#!/bin/sh
DESTINO=/home/app/bkp
pg_dump loja > $DESTINO/backup.sql
# cron: 0 3 * * *
```

caminho fixo no código

corrigido à mão na sessão: faltava compactar

nome sem data, sobrescreve o anterior

3h: escolha da IA, ninguém pediu

copia o que foi feito

serve de evidência

DESTILAR

o que a IA fez vira regra

sem conversa: vai direto para a regra

salvar em /home/app/bkp	✗ acaso empacotado
rodar às 3h	✗ escolha ao acaso virou regra
arquivo backup.sql	✗ o erro virou padrão

⇒ empacota o acaso

DECLARAR

o que deveria ter sido feito vira regra

conversa de brainstorm: isso fica ou muda?

- ✓ o caminho vem da variável BACKUP_DIR
- ✓ a data vai no nome do arquivo
- ✓ o horário é perguntado ao projeto
- ✓ sempre compacta (a correção feita à mão virou regra)

⇒ a sessão é evidência · a conversa é decisão

Uma skill ou duas

um COMO por skill

A RÉGUA DE GRANULARIDADE

GRANDE DEMAIS — certo — PEQUENA DEMAIS

mais larga ⇒ mais acionamento errado

mais estreita ⇒ mais esquecimento

```
name: banco-de-dados
description: Use para qualquer
  coisa de banco de dados.
```

sintoma: aciona onde não devia
um simples SELECT puxa a skill
inteira de backup

um COMO inteiro
em cada skill

nem puxa outro COMO,
nem deixa pedaço de fora

```
skills/
  nomear-arquivo-backup/
  compactar-backup/
  conferir-tamanho-backup/
```

sintoma: ninguém lembra que existe,
e nenhuma faz o trabalho inteiro

ONDE CORTAR: O TESTE APLICADO

o caso

gerar o backup
×
restaurar em
outro ambiente

Gerar o backup
se repete sem
restaurar?

sim

backup diário,
ninguém restaura

Restaurar
pressupõe um
backup pronto?

sim

sem backup, restaurar
não tem o que fazer

dois COMO
diferentes
⇒ duas
skills

```
backup-banco/SKILL.md
---
name: backup-banco
description: Use ao
  gerar ou agendar backup.
---
```

restaurar usa o arquivo
gerado pelo backup

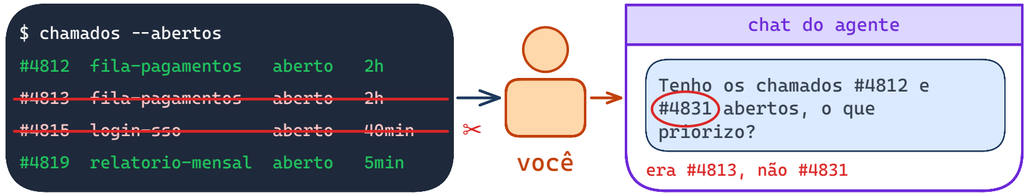
```
restaurar-banco/SKILL.md
---
name: restaurar-banco
description: Use ao
  restaurar um backup.
---
```

O que é o MCP

Um padrão para o agente alcançar o que está fora dele

Sem um padrão

A · Você no meio

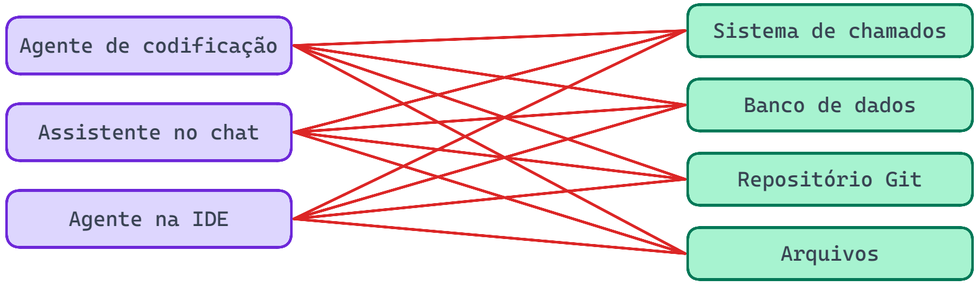


- linha cortada antes de perguntar
- janela gasta com o que não decide nada

você escolhe o que mostrar

- erro de transcrição que ninguém vê

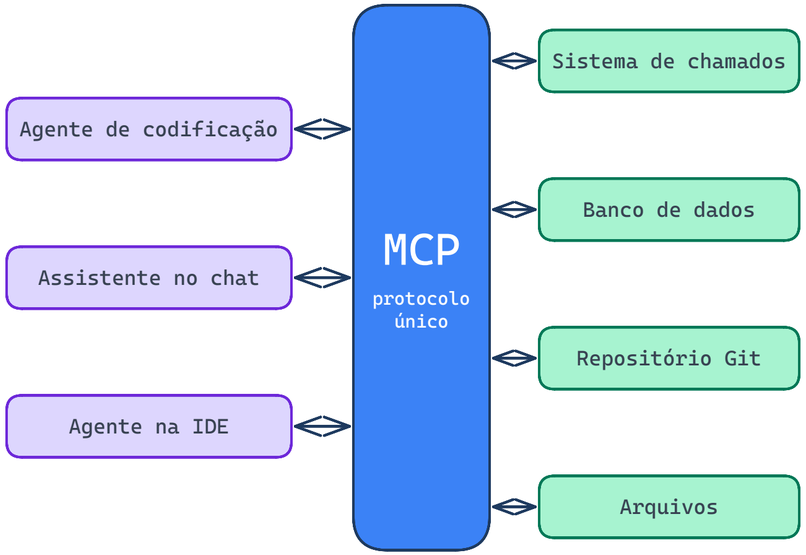
B · Uma integração para cada par



3 × 4 = 12 integrações feitas sob medida

Com MCP

- o agente consulta a fonte, em vez de receber o que você escolheu mostrar



3 + 4 = 7

cada agente fala o protocolo uma vez · cada serviço é exposto uma vez

MCP (Model Context Protocol): protocolo aberto que padroniza como aplicações de IA se conectam a dados e ferramentas externas

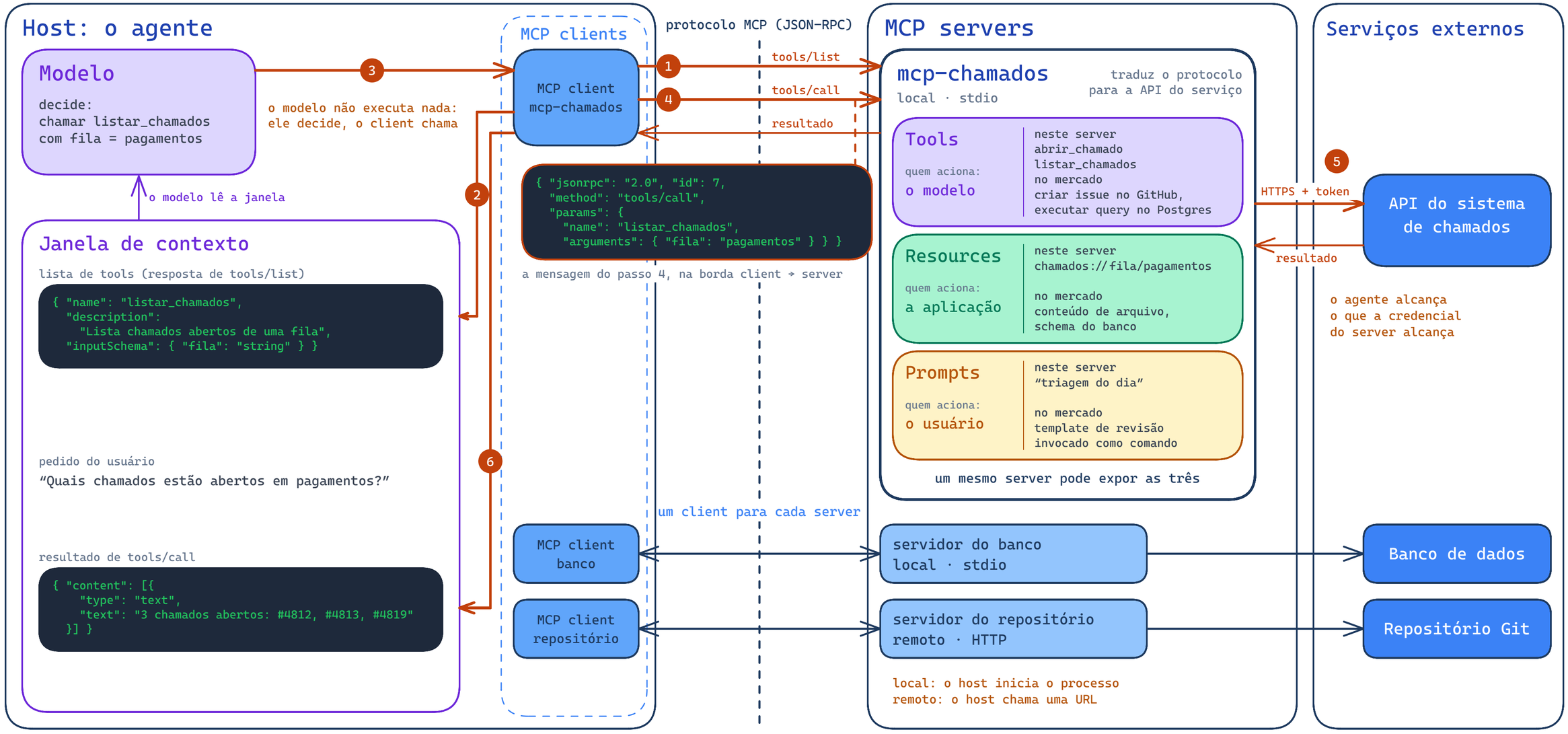
trazer contexto · ler o estado de um serviço

agir · executar uma ação nele

Arquitetura do MCP

Do pedido do modelo até o serviço externo, e de volta para a janela

1. host pede tools/list ao server
2. nome + descrição de cada tool entram na janela
3. o modelo decide: listar_chamados com fila = pagamentos
4. o client envia tools/call
5. o server chama a API do sistema de chamados
6. o resultado volta e entra na janela



Host → MCP client → MCP server → serviço externo

Construir e configurar um MCP

Quem constrói escreve o server · quem usa configura o host

server escrito → transporte → host configurado → conexão verificada

Quem constrói: o server

server.ts

```ts
const server = new McpServer({ name: "chamados", version: "1.0.0" });

server.registerTool(
  "abrir_chamado",
  {
    description: "Abre um chamado na fila indicada",
    inputSchema: z.object({ fila: z.string(), titulo: z.string() }),
  },
  async ({ fila, titulo }) => {
    const r = await api.post("/chamados", { fila, titulo });
    return {
      content: [{ type: "text", text: `Chamado #${r.id} aberto` }],
    };
  }
);

await server.connect(new StdioServerTransport());
```

nome
como o modelo chama a ferramenta

descrição
o modelo lê para decidir quando usar

schema
os argumentos que ele precisa preencher

handler
a chamada ao serviço externo
(o modelo nunca vê)

O que o modelo enxerga

abrir_chamado

Abre um chamado na fila indicada

argumentos: fila (texto), titulo (texto)

descrição vaga = ferramenta ignorada ou chamada na hora errada

Transporte

liga as duas pontas

stdio

server local:
o host inicia o processo e conversa pela entrada e saída padrão

entrada

saída

HTTP

server remoto:
o host chama uma URL

Quem usa: o host

configuração do host

```json
{
  "mcpServers": {
    "chamados": {
      "command": "npx",
      "args": ["mcp-chamados"],
      "env": { "CHAMADOS_TOKEN": "..." }
    },
    "repositorio": {
      "type": "http",
      "url": "https://mcp.exemplo.com/mcp"
    }
  }
}
```

command + args
como iniciar o server local

env
a credencial: define o que o server alcança

url
onde está o server remoto

o formato se repete entre ferramentas; o nome do arquivo muda

Escopo: onde a configuração vale

projeto
só este repositório, vai junto com o código

usuário
todo projeto seu, fica na sua máquina

Verificação

```
chamados     · conectado ✓
repositorio  · conectado ✓
```

listar é a evidência, não a esperança

O MCP é um usuário

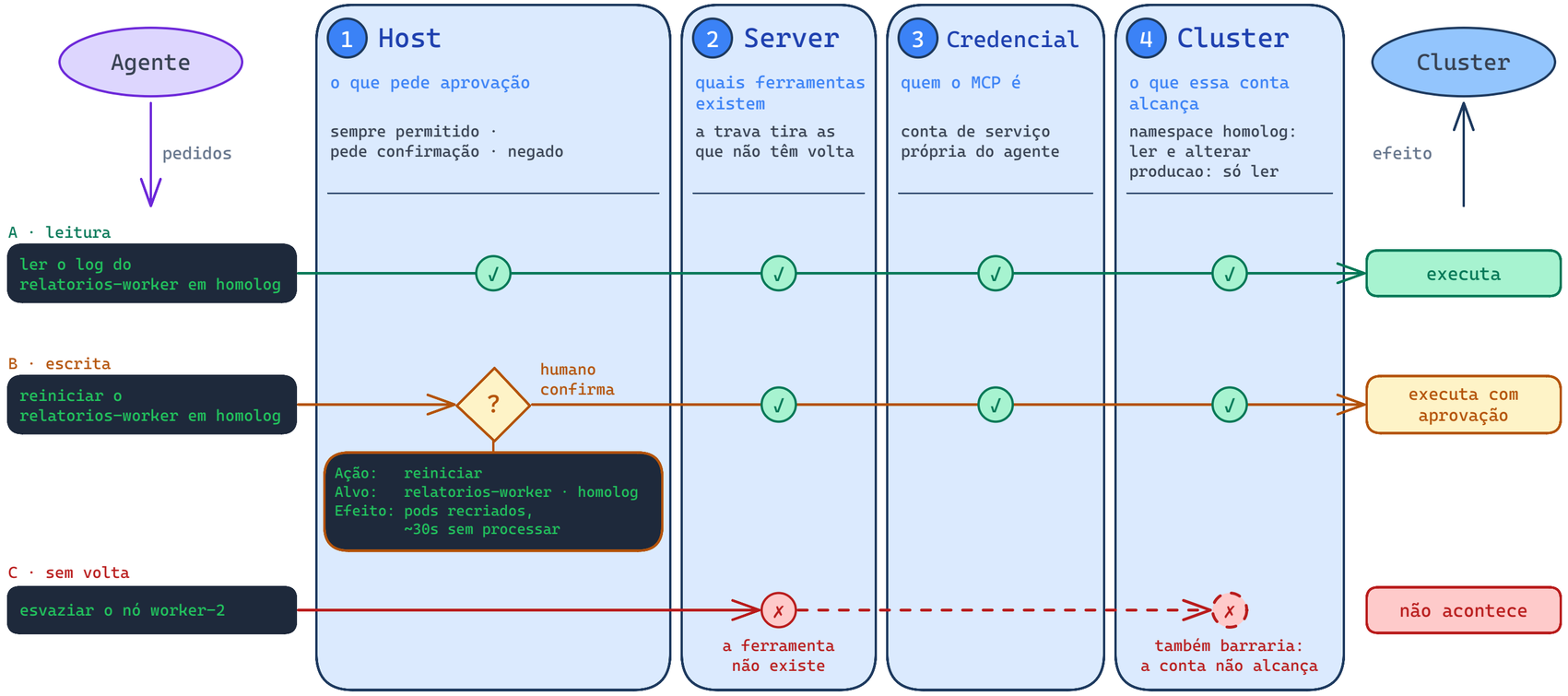
Mínimo privilégio · acesso só ao que deve · execução validada

Trate como um usuário

Uma pessoa nova no time	O agente pelo MCP
conta própria	conta de serviço própria, nunca a sua
acesso só ao que a função pede	só os recursos e ações da tarefa
ação crítica passa por revisão	escrita passa por confirmação
acesso revisto quando a função muda	escopo revisado, não esquecido

o agente com a sua credencial de administrador alcança tudo o que você alcança

Camadas de permissão



cada camada barra sozinha

Níveis de permissão

risco crescente →

Leitura

obter · descrever · ler log

liberado

Escrita

aplicar · criar · escalar · reiniciar

com confirmação: a ação · o alvo · o efeito esperado

Sem volta

apagar · remover · esvaziar nó

fora da superfície do agente

o que o agente não alcança, ele não erra · ler o comando e apertar sim não é revisão